ROOTCAUSE SERVER

13 · Despliegue y operación

Documento: 13-deployment-and-operations.md **Versión analizada:** 0.2.0 **Commit:** 767a67b
Fecha del análisis: 2026-08-26

CONTENIDO

- Entornos
- Proceso de construcción
- Empaquetado
  - Contenedor
  - Endurecimiento del despliegue en Compose
  - Servicios del sistema
- Infraestructura
- CI/CD
  - Integración continua
  - Auditoría de seguridad
  - Publicación
  - Página del producto
- Migraciones
- Registros
- Métricas y monitoreo
  - Sondas de disponibilidad
- Alertas
- Respaldos
- Recuperación
- Reversión
- Mantenimiento
  - Rutina semanal
  - Rutina mensual
  - Con cada versión
  - Actualización sin interrupción

> Entornos, construcción, empaquetado, contenedores, infraestructura, CI/CD, publicación, migraciones, registros, métricas, monitoreo, alertas, respaldos, recuperación, reversión y mantenimiento.
>
> Complementa a `docs/INSTALLATION.md` y `docs/HARDENING.md`.

Entornos

Entorno	Cómo se levanta	Enlace	Autenticación	Base de datos
Desarrollo	`scripts/dev.sh` o `cargo run`	`127.0.0.1:8080`	Token generado, o `--insecure-dev-mode`	Archivo local

Entorno	Cómo se levanta	Enlace	Autenticación	Base de datos
CI / pruebas	`cargo test` y `scripts/smoke.sh`	`127.0.0.1:18080`	Token fijo del guion	`sqlite::memory:` y temporal
Producción, binario	systemd o launchd	Loopback, tras proxy	Token obligatorio	`/var/lib/rootcause/rootcause.db`
Producción, contenedor	`docker compose up -d`	`0.0.0.0:8080` en el contenedor, publicado solo en `127.0.0.1` del anfitrión	Token obligatorio	Volumen `rootcause-data`

No hay entorno de *staging* definido en el repositorio. *No identificado.*

Proceso de construcción

Perfil	Comando	Características
`dev`	`cargo build --workspace`	Sin optimizar, con símbolos
`release`	`cargo build --release --locked`	LTO fino, `codegen-units = 1`, símbolos eliminados, `panic = abort`
`dist`	`cargo build --profile dist --locked`	Hereda de `release` y sube a LTO completo. Es el perfil de publicación

`panic = abort` es una decisión de seguridad además de de tamaño: sin desenrollado, un pánico termina el proceso en lugar de dejarlo en un estado parcialmente deshecho.

Empaquetado

Contenedor

`Dockerfile` en dos etapas:

1. **Constructor** sobre `rust:1.95-bookworm`. Copia primero los manifiestos y ejecuta `cargo fetch --locked` con `src/` de relleno, para que una edición del motor de detección no recompile todo el árbol de dependencias. Después copia `crates/` y `console/`, compila `rootcause-server` en `release` y le quita los símbolos.
2. **Ejecución** sobre `debian:bookworm-slim`. Instala solo `ca-certificates`, crea el usuario del sistema `rootcause` con UID **10001** y su directorio de datos, copia el binario y cambia a ese usuario.

Etiquetas OCI: título, descripción, `source` y `licenses`.

La imagen contiene un binario y nada más. La consola va compilada dentro, así que no hay directorio de recursos que montar mal ni servidor de archivos que endurecer.

Endurecimiento del despliegue en Compose

`compose.yml`:

Ajuste	Valor	Para qué
`read_only`	`true`	Sistema de archivos inmutable salvo el volumen
`tmpfs`	`/tmp:size=16m,noexec,nosuid,nodev`	Espacio temporal sin ejecución
`security_opt`	`no-new-privileges:true`	Impide elevar privilegios

Ajuste	Valor	Para qué
`cap_drop`	`ALL`	Ninguna capacidad de Linux
`ports`	`127.0.0.1:8080:8080`	Quien deba llegar de fuera pasa por el proxy
`volumes`	`rootcause-data:/var/lib/rootcause`	Persistencia de la base
`healthcheck`	`bash -c 'echo > /dev/tcp/127.0.0.1/8080'`	Comprueba el escucha real **sin añadir** `curl` a una imagen que no lo necesita
`logging`	`json-file`, 10 MB × 5	Rotación acotada
`restart`	`unless-stopped`	Reinicio automático

El token se pasa con `${ROOTCAUSE_API_TOKEN:?…}`: si falta, Compose **se niega a arrancar** con un mensaje que dice cómo generarlo.

Servicios del sistema

Archivo	Plataforma
`packaging/linux/rootcause-server.service`	systemd
`packaging/linux/rootcause-agent.service`	systemd
`packaging/macos/com.rootcause.server.plist`	launchd
`packaging/macos/com.rootcause.agent.plist`	launchd
`packaging/windows/rootcause-server.iss`	Inno Setup

Endurecimiento de la unidad del servidor:

```
User=rootcause
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
ProtectSystem=strict
ReadWritePaths=/var/lib/rootcause
CapabilityBoundingSet=
LockPersonality=true
MemoryDenyWriteExecute=true
RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX
SystemCallArchitectures=native
```

La configuración se lee de `/etc/rootcause/server.env`, que **debe pertenecer a `root` con modo `0600`**: contiene el token.

> El instalador de Windows **no se compiló** durante este análisis. Su contenido se revisó, pero la comprobación de que produce un instalador funcional *requiere validación*.

Infraestructura

No hay infraestructura como código: ni Terraform, ni Ansible, ni manifiestos de Kubernetes. *No identificado*, y es coherente con el alcance declarado —un binario, un archivo y un proxy delante—.

Requisitos mínimos del anfitrión, *inferencia basada en el código*:

Recurso	Estimación	Razón
CPU	1 núcleo	El trabajo por ingesta es milisegundos de reglas puras
Memoria	128–256 MB	Binario Rust sin recolector, más el pool de SQLite y hasta 20 000 entradas de perímetro
Disco	Crece con activos × frecuencia × retención	*Requiere validación* con la flota real
Red	Un puerto entrante	—

CI/CD

Integración continua

15 trabajos, detallados en 12 · Pruebas y calidad. Lo distintivo es la **puerta final**: falla si algún control quedó en `skipped` o `cancelled`, no solo si falló.

Auditoría de seguridad

`security.yml` en cada push y PR, más cada lunes a las 06:17 UTC: avisos publicados, secretos en el histórico (`gitleaks`), dependencias y configuración del repositorio, workflows de Actions e inventario de dependencias (SBOM CycloneDX).

Publicación

`release.yml`, disparado por una etiqueta `v*` o manualmente. Tres fases:

1 · Verificar antes de publicar. Formato, Clippy, pruebas y una comprobación específica: **la versión de la etiqueta debe coincidir con la de `Cargo.toml`**. Publicar `v0.3.0` con un manifiesto que dice `0.2.0` hace fallar el workflow.

2 · Construir, en cuatro objetivos:

Nombre	Sistema del runner	Objetivo
`linux-x86_64`	`ubuntu-latest`	`x86_64-unknown-linux-gnu`
`linux-aarch64`	`ubuntu-24.04-arm`	`aarch64-unknown-linux-gnu`
`windows-x86_64`	`windows-latest`	`x86_64-pc-windows-msvc`
`macos-aarch64`	`macos-latest`	`aarch64-apple-darwin`

Sin caché de compilación, a propósito. El comentario del workflow lo explica: un artefacto que se publica y se firma no debe poder heredar nada de una caché envenenable. Los permisos del trabajo incluyen `id-token: write` y `attestations: write` para la atestación de procedencia.

3 · Publicar. Descarga los artefactos, genera el SBOM con `scripts/sbom.py`, consolida `SHA256SUMS.txt`, extrae del `CHANGELOG.md` la sección de esa versión con `awk` y crea la publicación de GitHub con los `.tar.gz`, los `.sha256`, las sumas y el SBOM.

Las notas incluyen siempre cómo verificar lo descargado:

```
sha256sum --check SHA256SUMS.txt
```

```
gh attestation verify rootcause-linux-x86_64.tar.gz --repo vladimiracunadev-create/rootcause-server
```

> **La atestación de procedencia y la firma no se verificaron en este análisis**, porque exigiría ejecutar el workflow y consultar la API de GitHub. El workflow declara los permisos correctos y las notas documentan la comprobación. *Requiere validación* al publicar la próxima versión.

Página del producto

`deploy-landing.yml` publica `landing/` en GitHub Pages cuando cambia en `main`, tras comprobar que **la página no carga recursos de otro origen**.

Migraciones

Se aplican **al arrancar**, dentro de `Database::connect`, con `sqlx::migrate!("./migrations")`, que las embebe en el binario. No hay comando de migración separado ni paso manual.

Situación	Comportamiento
Base nueva	Se crea y se aplican `0001` y `0002`
Base de 0.1.0	Se aplica solo `0002`; todo lo añadido es nullable o tiene valor por omisión
Base al día	No se aplica nada
Migración fallida	El arranque falla con `database migration failed`
Reversión	**No existe.** No hay migraciones descendentes

Volver a una versión anterior del binario exige restaurar un respaldo previo a la actualización. *Requiere validación* del procedimiento en el entorno real.

Registros

Aspecto	Valor
Biblioteca	`tracing` con `tracing-subscriber`
Destino	`stdout`
Formato	Compacto por omisión; JSON con `ROOTCAUSE_JSON_LOGS=true`
Filtro	`ROOTCAUSE_LOG`, por omisión `rootcause_server=info,tower_http=warn`
Trazas HTTP	`TraceLayer` de `tower-http`
Rotación	La hace el recolector: `journald`, `json-file` de Docker o el agregador

Eventos que conviene vigilar:

Nivel	Mensaje	Significado
`warn`	*modo de desarrollo inseguro: la API no exige token…*	El servidor corre sin autenticación
`warn`	*el servidor escucha fuera de loopback…*	Enlace público; debe haber TLS delante
`warn`	*dirección bloqueada tras repetidos fallos de autenticación*	Alguien está adivinando el token
`warn`	*no se pudo evaluar el silencio de los agentes*	Fallo del vigilante

Nivel	Mensaje	Significado
`warn`	*no se pudo aplicar la retención*	La base no aceptó la poda
`error`	*la solicitud falló dentro del servidor*	Error interno; **la causa está aquí y no en la respuesta**
`info`	*RootCause Server listo*	Arranque, con toda la configuración efectiva
`info`	*retención aplicada*	Poda con filas eliminadas

Métricas y monitoreo

`GET /metrics` en formato Prometheus, **autenticado**. Las once métricas están en 05 · Referencia técnica.

Ejemplo de raspado:

```
scrape_configs:
  - job_name: rootcause
    metrics_path: /metrics
    authorization:
      type: Bearer
      credentials_file: /etc/prometheus/rootcause.token
    static_configs:
      - targets: ["127.0.0.1:8080"]
```

Sondas de disponibilidad

Endpoint	Qué comprueba	Uso
`GET /healthz`	Que el proceso responde	*Liveness*
`GET /readyz`	Que el almacenamiento responde (`SELECT 1`)	*Readiness*

Ninguno requiere token, así que un orquestador puede consultarlos sin credenciales.

Alertas

El sistema no envía alertas. No hay correo, ni webhook, ni notificación saliente. Está declarado como planificado en `docs/ROADMAP.md`.

Alertas recomendadas sobre las métricas expuestas —*propuesta de este análisis, no configuración existente*—:

Alerta	Expresión	Por qué
Plano de control caído	`up{job="rootcause"} == 0`	Sin sensor no hay defensa
Incidente crítico abierto	`rootcause_incidents_critical > 0`	Requiere atención inmediata
Postura degradada	`rootcause_posture_score < 70`	Nota C o peor
Agentes desaparecidos	`rootcause_assets_total - rootcause_assets_online > 0`	Puede ser mantenimiento… o manipulación
Servicios expuestos en aumento	`increase(rootcause_exposed_services[1h]) > 0`	Alguien publicó algo
Presión sobre el propio plano de control	`rootcause_perimeter_locked_sources > 0`	Alguien adivina el token

Respaldos

Estado en el repositorio: `scripts/backup.sh`, con su equivalente `scripts/backup.ps1`.

```
bash scripts/backup.sh /var/lib/rootcause/rootcause.db /respaldo
```

Copia con `.backup` de SQLite —seguro con el servidor escribiendo—, comprueba la copia con `PRAGMA integrity_check`, contrasta el número de filas de `incidents`, `audit_log` y `assets` contra el origen, y conserva las N más recientes (`ROOTCAUSE_BACKUP_KEEP`, 14 por omisión). Falla con código 1 si cualquier comprobación no cuadra.

Para ejecutarlo cada noche con systemd:

```
[Unit]
Description=Respaldo verificado de RootCause

[Service]
Type=oneshot
User=rootcause
ExecStart=/usr/local/bin/rootcause-backup.sh /var/lib/rootcause/rootcause.db /respaldo
```

Lo que el script no hace, y sigue siendo responsabilidad de quien opera: programarlo, llevarse las copias fuera del anfitrión y **probar una restauración de verdad**. Un respaldo que nunca se restauró sigue sin ser un respaldo.

Qué respaldar, por orden de importancia:

Elemento	Frecuencia sugerida	Motivo
`rootcause.db`	Diaria	Contiene incidentes y auditoría, que **no** se pueden regenerar
Archivo de política	Con cada cambio	Debería estar en control de versiones
`/etc/rootcause/*.env`	Con cada cambio	**Contiene el token**; guardar cifrado

Recuperación

Escenario	Procedimiento	Pérdida
Proceso caído	`systemctl restart rootcause-server`	El estado del perímetro; los bloqueos en curso se levantan
Base corrupta	Restaurar el último respaldo	Todo lo posterior al respaldo
Anfitrión perdido	Reinstalar binario, restaurar base y `.env`, reapuntar los agentes	Lo no respaldado
Token comprometido	Generar uno nuevo, actualizar el servidor y **todos** los agentes	Sin pérdida de datos; **hay interrupción**, porque no existe rotación con solapamiento
Actualización fallida	Restaurar binario anterior **y** base anterior	Sin migración descendente

Los agentes se recuperan solos. Su reintento exponencial vuelve al intervalo normal en cuanto el servidor responde. Lo que se pierde son los ciclos no entregados: **no hay cola local**.

Reversión

Elemento	Reversible	Cómo
Binario	Sí	Reinstalar la versión anterior
Esquema de base de datos	**No**	Sin migraciones descendentes; hay que restaurar respaldo
Configuración	Sí	Revertir el `.env` y reiniciar
Política de detección	Sí	Volver al archivo anterior y reiniciar
Imagen de contenedor	Sí	Fijar la etiqueta anterior
Publicación de GitHub	Sí	Marcarla como borrador o eliminarla

El orden importa al revertir una actualización de esquema: primero detener el servicio, después restaurar la base, y solo entonces arrancar el binario antiguo. Arrancar un binario `0.1` contra una base ya migrada a `0.2` no está probado. *Requiere validación.*

Mantenimiento

Rutina semanal

1. Revisar la ejecución programada del workflow de seguridad (lunes 06:17 UTC).
2. Revisar los PR de Dependabot.
3. Comprobar que ningún activo lleva demasiado tiempo `offline`.
4. Revisar `GET /api/v1/threats`, incluidos los contadores de defensa del propio plano de control.

Rutina mensual

1. Verificar que el respaldo existe **y que restaura**.
2. Revisar el crecimiento del archivo SQLite frente a la retención configurada.
3. Revisar incidentes reconocidos que llevan mucho abiertos.
4. Comprobar `hardening` en `GET /api/v1/status`: nadie debería haber desactivado la autenticación sin querer.

Con cada versión

1. Leer el `CHANGELOG.md`.
2. Respaldar antes de actualizar.
3. Actualizar el servidor **antes** que los agentes: el protocolo `1.1` admite agentes `1.0`, pero no al revés.
4. Verificar sumas y atestación de los artefactos descargados.
5. Comprobar `GET /api/v1/status` tras arrancar.

Actualización sin interrupción

No es posible hoy. Una sola instancia, sin alta disponibilidad. La interrupción dura lo que tarde el reinicio; los agentes reintentan solos, así que el efecto práctico es un hueco en la telemetría. Recogido como límite declarado en `docs/CAPABILITIES.md`.

Anterior: 12 · Pruebas y calidad · **Siguiente:** 14 · Solución de problemas · **Índice:** README